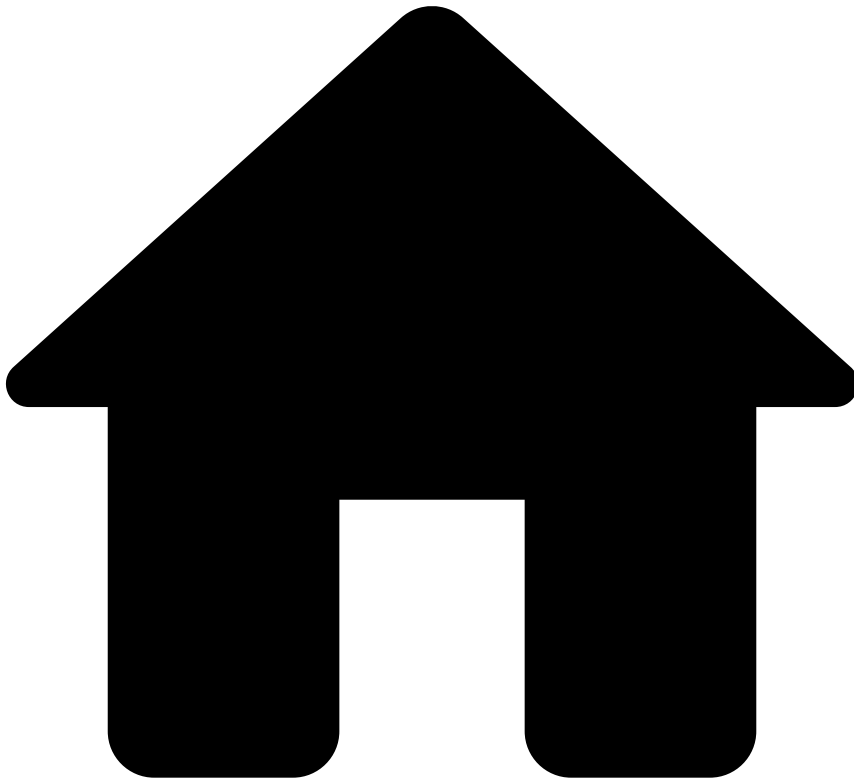


•



- [API Resources](#)
- [Additional Resources](#)
- Historical Data Loading

On this page

Historical Data Loading

Was this page helpful?



Historical data loading allows you to send past data that can be used to backfill previous events. This is done via API and does not compromise any event processing that is in progress.

This page explains the historical data loading's end-to-end process.

Purpose

Historical data loading is useful in the following situations:

- **Pre-activation:** During activation, historical data loading prepares the system by ingesting events and calculating the initial profiles.
- **Post-activation:**
 - Tenant onboarding: BaaS or multi-tenanted clients may onboard new tenants after the go-live. These tenants bring their own set of historical data that needs to be loaded into an already running environment.
 - Product expansion: Clients may decide to integrate additional use cases from Feedzai's portfolio or support new event types.
- **Recovery:** In case there are integration issues or other external factors, event data may be missing, which may lead to unalerted events. Data can be injected at a later stage as mitigation.



info

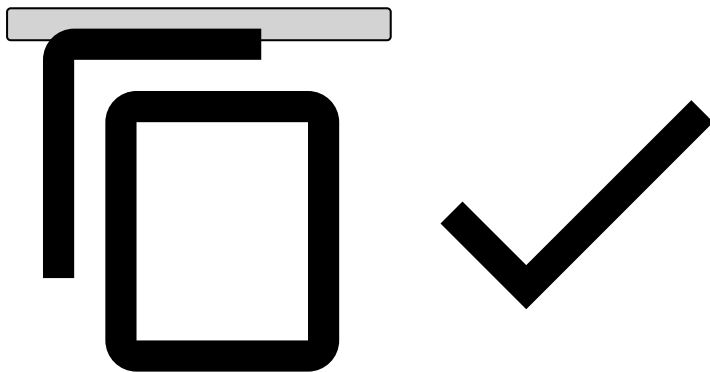
Historical data loading can be done manually or in an automated way. Either way, it needs to be performed by your IT team or similar integrator role.

See below what needs to be considered to load historical data.

Data preparation📄

Ensure that the historical data's format is consistent with normal API requests. Convert it accordingly, if necessary.

```
curl --location --request PUT 'http://localhost:8083/api/referenceData/customers/1791457' \
--header 'Content-Type: application/json' \
--header 'Backfill: true' \
--header 'Authorization: Bearer {token}' \
--data-raw '{
  "customer_id": "1791457",
  "customer_active": "true",
  "customer_alias_status": "active",
  "customer_segment": "Retail",
  "customer_name": "Anthony Manoel",
  "customer_email": "[email protected]",
  "customer_phone": "+351276333728",
  "customer_phone_val_code": "T",
  "customer_citizen_id_number": "3254756464",
  "customer_passport_number": "7856875675",
  "customer_driver_license": "76576865",
  "customer_ssn": "543654356",
  "customer_tax_number": "76587656785",
  "customer_registration_date": 1420074061000,
}'
```

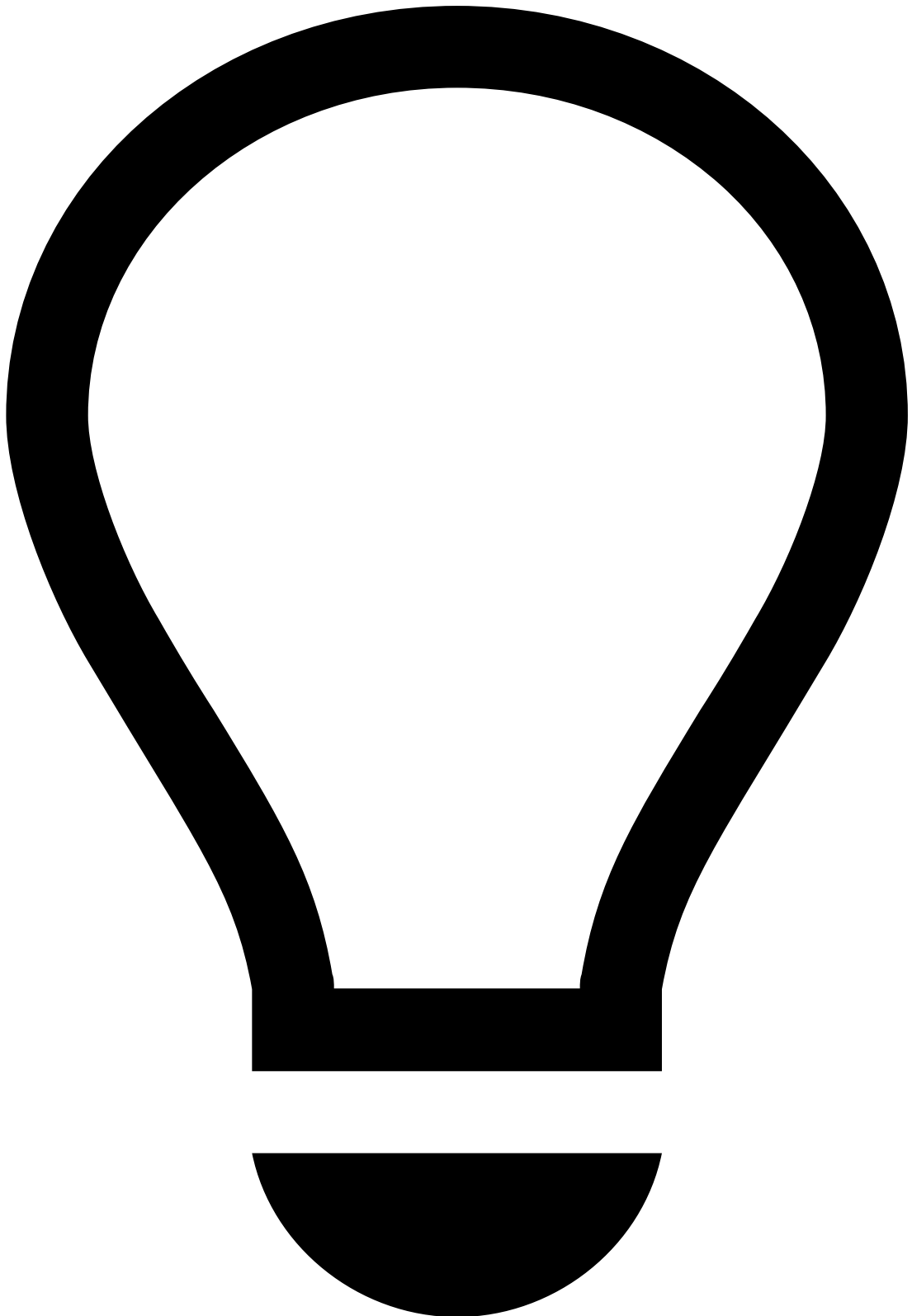


Event backfilling📄

Historical data requests are injected through the usual [endpoints](#).

Such requests must include a `Backfill` HTTP request header set to `true`. Note that omitting the header or setting the value to `false` will result in the request being processed in the regular way (i.e. not backfill).

Note that backfilling does not change an event's date and time, i.e. the original event timestamp (`event_occurred_at`) remains unchanged.



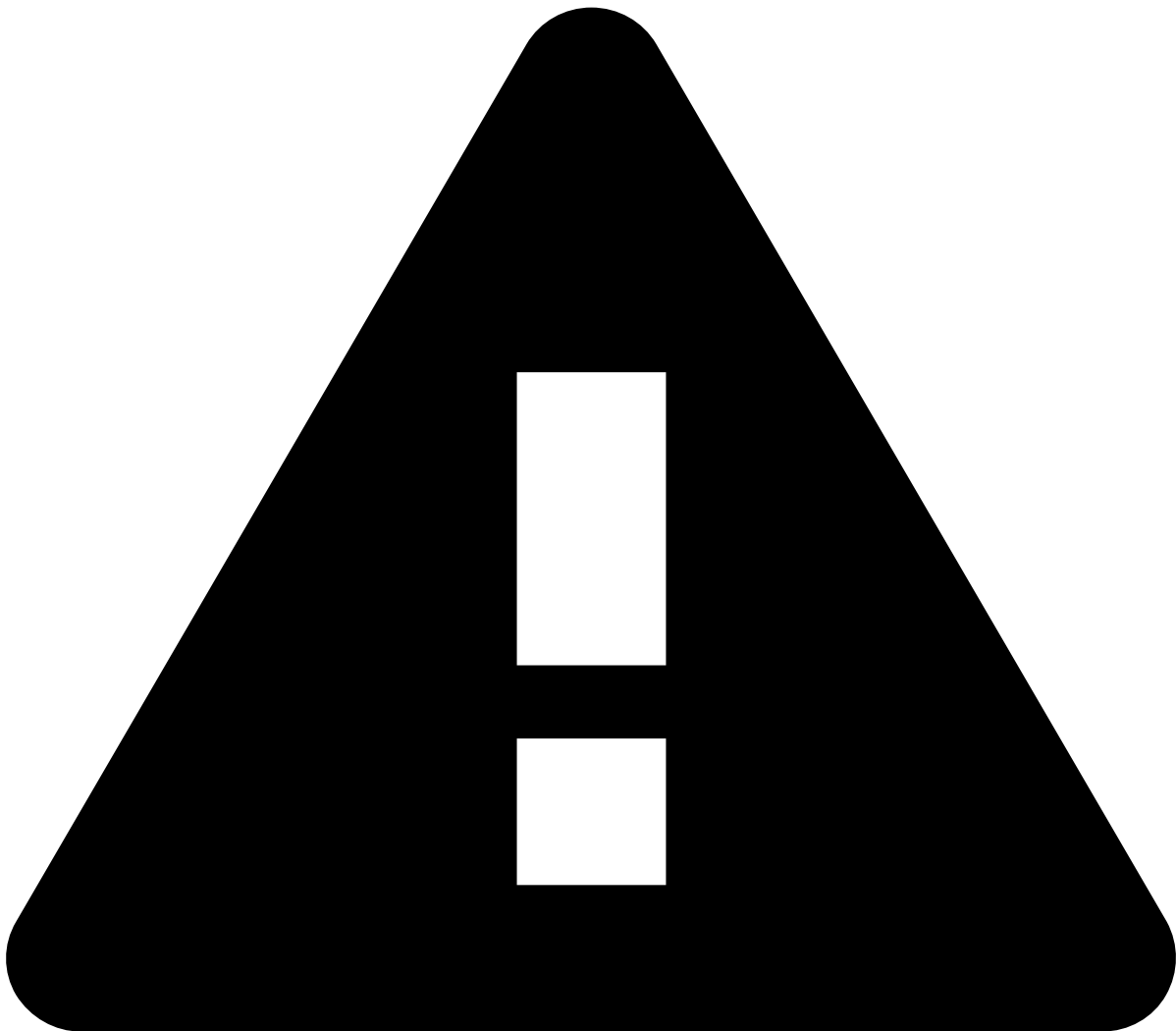
Recommendation

Send event data from most recent to oldest for efficiency reasons. This way, the backfilled data can more quickly mitigate gaps by contributing to the short-window profiles first.

To minimize the impact of backfilling data in ongoing processing latencies, historical data loading is configured with the following properties:

1. **Asynchronous processing:** Backfill requests will be processed asynchronously. This is enabled at endpoint level. When enabled, incoming events are placed in a queue, which is then consumed at a controlled, configurable rate. This allows the system to throttle received events.

2. **Backfill rate limiter:** The system will discard backfill requests if the number of requests per second exceeds a defined threshold. Requests exceeding the rate limit receive an HTTP 429 (Too Many Requests) response. This value is determined in each client based on the contracted tier and volume.
3. **Lower priority:** Backfill requests will be assigned a lower priority than regular events.

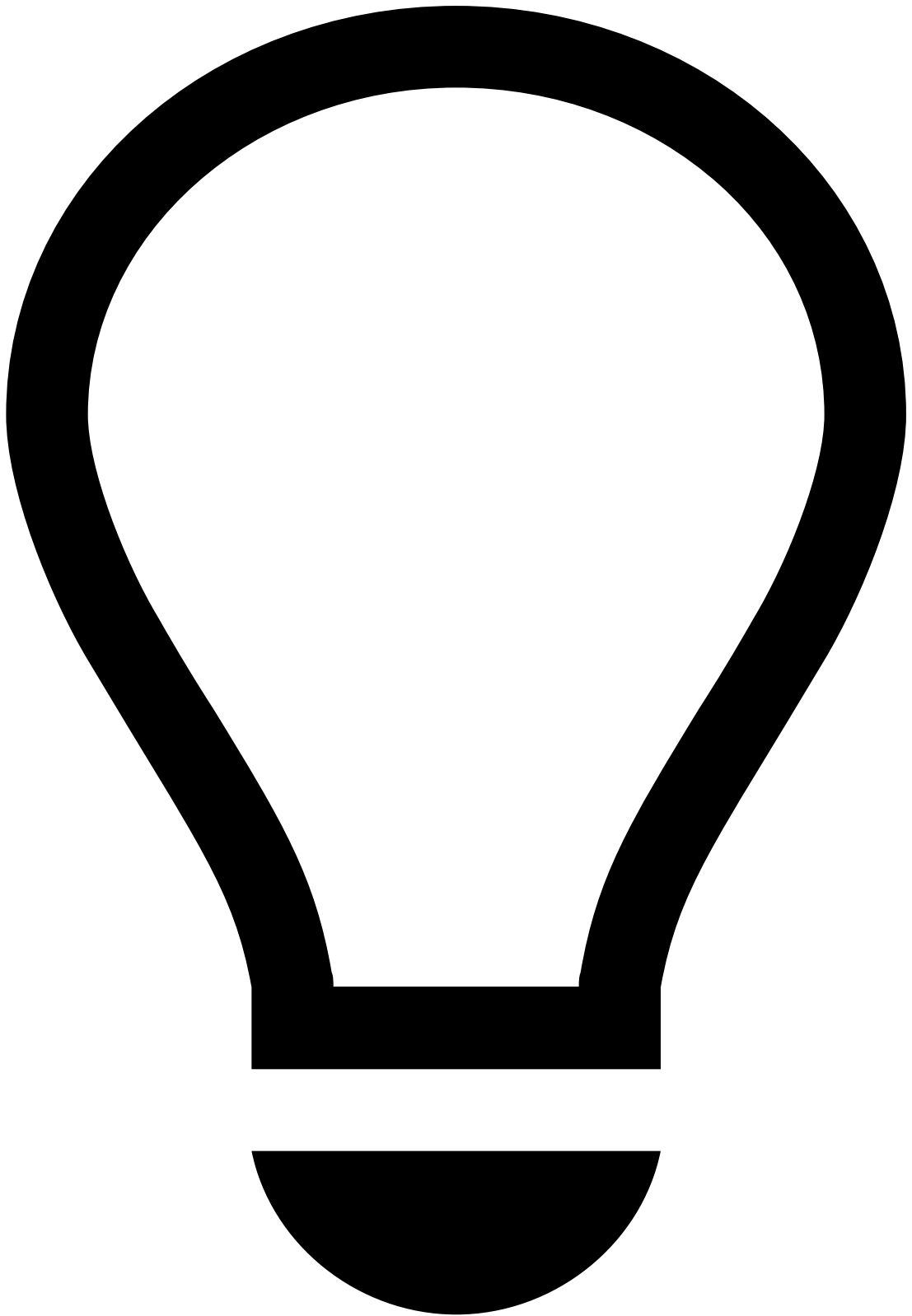


Historical data loading setup

Reach out to Feedzai for the initial feature configuration, so that the above properties are configured to best suit your system deployment.

Metrics update

Once the data injection has been completed, a Railgun backfill must be manually triggered to update the metrics.



tip

Refer to the Pulse documentation to learn how to [manually trigger a Railgun backfill](#).